6. To Create Process Escape Sequences, Handle Escaped Spaces, Escape Special Symbols, Preserve Characters, Validate Parser Output, Test Complex Inputs.

Code:



```
ketana_sindhu@KetaZone: ~/
  GNU nano 8.7.1
#include <stdio.h>
#include <string.h>

#define MAX_INPUT 200
#define MAX_OUTPUT 200

/*
 * Process escape sequences.
 *
 * Supported:
 *    \\   => backslash
 *    \    => escaped space
 *    \n   => newline
 *    \t   => tab
 *    \&   => &
 *    \|   => |
 *    \<   => <
 *    \>   => >
 *    \$   => $
 *    \\   => \
 */

int processEscapes(const char *input, char *output) {

    int i = 0;
    int j = 0;

    while (input[i] != '\0' && j < MAX_OUTPUT - 1) {

        /* Normal character */
        if (input[i] != '\\') {

            output[j++] = input[i];
            i++;
        }

        /* Escape sequence */
        else {

            i++;

            /* Backslash at end of input */
            if (input[i] == '\0') {
                printf("Error: Incomplete escape sequence.\n");
                return 0;
            }

            switch (input[i]) {

                case 'n':
                    output[j++] = '\n';
                    break;

                case 't':
                    output[j++] = '\t';
                    break;

                case ' ':
                    output[j++] = ' ';
                    break;
```



```
  GNU nano 8.7.1
                case ' ':
                    output[j++] = ' ';
                    break;

                case '&':
                    output[j++] = '&';
                    break;

                case '|':
                    output[j++] = '|';
                    break;

                case '<':
                    output[j++] = '<';
                    break;

                case '>':
                    output[j++] = '>';
                    break;

                case '$':
                    output[j++] = '$';
                    break;

                case '\\':
                    output[j++] = '\\';
                    break;

                default:
                    /*
                     * Preserve unknown escaped characters.
                     */
                    output[j++] = input[i];
                    break;
            }

            i++;
        }
    }

    output[j] = '\0';

    return 1;
}

/* Display characters including special characters */
void displayOutput(const char *output) {

    printf("\nProcessed Output:\n");
    printf("-----------------------------------\n");

    for (int i = 0; output[i] != '\0'; i++) {

        if (output[i] == '\n')
            printf("[NEWLINE]\n");

        else if (output[i] == '\t')
            printf("[TAB]");

        else
```

```
  GNU nano 8.7.1
/* Display characters including special characters */
void displayOutput(const char *output) {

    printf("\nProcessed Output:\n");
    printf("--------------------------------\n");

    for (int i = 0; output[i] != '\0'; i++) {

        if (output[i] == '\n')
            printf("[NEWLINE]\n");

        else if (output[i] == '\t')
            printf("[TAB]");

        else
            printf("%c", output[i]);
    }

    printf("\n--------------------------------\n");
}

/* Validate parser output */
void validateOutput(const char *output) {

    if (strlen(output) == 0) {
        printf("Parser Output: Empty.\n");
    }
    else {
        printf("Parser Output: Valid.\n");
    }
}

int main() {

    char input[MAX_INPUT];
    char output[MAX_OUTPUT];

    printf("Escape Sequence Parser\n");
    printf("----------------------\n");

    printf("Enter input: ");

    fgets(input, sizeof(input), stdin);

    /* Remove newline added by fgets */
    input[strcspn(input, "\n")] = '\0';

    /* Process escape sequences */
    if (!processEscapes(input, output)) {
        return 1;
    }

    /* Display result */
    displayOutput(output);

    /* Validate result */
    validateOutput(output);

    return 0;
```

Result:

```
sanjana3027@DESKTOP-QJF5AGU:~/SKILL$ nano Skill6a.c
sanjana3027@DESKTOP-QJF5AGU:~/SKILL$ gcc -Wall -Wextra Skill6a.c -o s6a
sanjana3027@DESKTOP-QJF5AGU:~/SKILL$ ./s6a
Escape Sequence Parser
======================
Enter input: Hello\ World

Processed Output:
--------------------------------
Hello World
--------------------------------
Parser Output: Valid.
sanjana3027@DESKTOP-QJF5AGU:~/SKILL$ Hello\nWorld
HellonWorld: command not found
sanjana3027@DESKTOP-QJF5AGU:~/SKILL$
```

To Create Child Processes, Execute Programs, Handle Execution Errors, Pass Arguments, Manage Parent Process, Test Command Launching.

```
  GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {

    pid_t pid;
    int status;

    printf("Parent Process Started\n");
    printf("Parent PID: %d\n", getpid());

    // Create child process
    pid = fork();

    if (pid < 0) {

        // Handle fork error
        perror("fork failed");
        return 1;
    }

    else if (pid == 0) {

        // Child process
        printf("\nChild Process Created\n");
        printf("Child PID: %d\n", getpid());
        printf("Parent PID: %d\n", getppid());

        // Execute program with arguments
        printf("Executing ls command...\n");

        execlp("ls", "ls", "-l", NULL);

        // This executes only if execlp fails
        perror("Execution failed");
        exit(1);
    }

    else {

        // Parent process
        printf("\nParent waiting for child...\n");

        // Wait for child process
        if (waitpid(pid, &status, 0) == -1) {

            perror("waitpid failed");
            return 1;
        }

        // Check child exit status
        if (WIFEXITED(status)) {

            printf("\nChild process completed.\n");
            printf("Child exit status: %d\n",
                   WEXITSTATUS(status));
        }
```

```
    else {

        // Parent process
        printf("\nParent waiting for child...\n");

        // Wait for child process
        if (waitpid(pid, &status, 0) == -1) {

            perror("waitpid failed");
            return 1;
        }

        // Check child exit status
        if (WIFEXITED(status)) {

            printf("\nChild process completed.\n");
            printf("Child exit status: %d\n",
                   WEXITSTATUS(status));
        }

        else {
            printf("Child process terminated abnormally.\n");
        }
    }

    printf("\nParent Process Completed\n");

    return 0;
}
```

Result:

```
sanjana3027@DESKTOP-QJF5AGU:~/SKILL$ nano Skill6b.c
sanjana3027@DESKTOP-QJF5AGU:~/SKILL$ gcc -Wall -Wextra Skill6b.c -o s6b
sanjana3027@DESKTOP-QJF5AGU:~/SKILL$ ./s6b
Parent Process Started
Parent PID: 1132

Parent waiting for child...

Child Process Created
Child PID: 1133
Parent PID: 1132
Executing ls command...
total 184
-rwxr-xr-x 1 sanjana3027 sanjana3027 16744 Oct 13 03:10 Skill3a
-rw-r--r-- 1 sanjana3027 sanjana3027  3576 Oct 13 03:10 Skill3a.c
-rwxr-xr-x 1 sanjana3027 sanjana3027 16416 Oct 13 03:27 Skill4a
-rw-r--r-- 1 sanjana3027 sanjana3027  3414 Oct 13 03:27 Skill4a.c
-rwxr-xr-x 1 sanjana3027 sanjana3027 16624 Oct 13 03:28 Skill4b
-rw-r--r-- 1 sanjana3027 sanjana3027  3469 Oct 13 03:28 Skill4b.c
-rwxr-xr-x 1 sanjana3027 sanjana3027 16336 Oct 13 03:31 Skill5a
-rw-r--r-- 1 sanjana3027 sanjana3027  2563 Oct 13 03:30 Skill5a.c
-rwxr-xr-x 1 sanjana3027 sanjana3027 16464 Oct 13 03:32 Skill5b
-rw-r--r-- 1 sanjana3027 sanjana3027  3528 Oct 13 03:32 Skill5b.c
-rw-r--r-- 1 sanjana3027 sanjana3027  2339 Oct 13 03:35 Skill6a.c
-rw-r--r-- 1 sanjana3027 sanjana3027  1346 Oct 13 03:37 Skill6b.c
-rw-r--r-- 1 sanjana3027 sanjana3027    12 Oct 13 03:27 output.txt
-rwxr-xr-x 1 sanjana3027 sanjana3027 16336 Oct 13 03:35 s6a
-rwxr-xr-x 1 sanjana3027 sanjana3027 16432 Oct 13 03:37 s6b
-rwxr-xr-x 1 sanjana3027 sanjana3027 16176 Aug 24 05:55 skill2
```